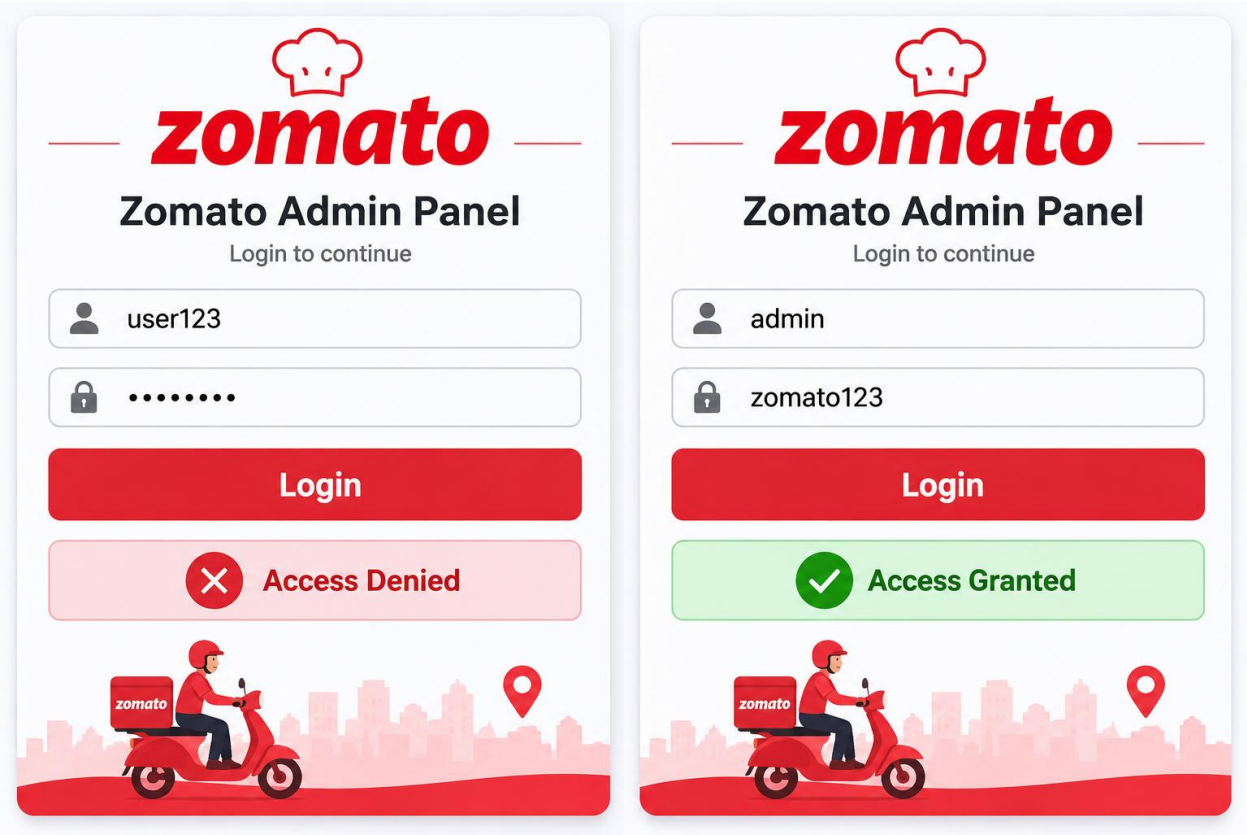
1)Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Ans.



2)Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Use a dictionary to map roles to allowed resources.

Ans. Python Program: Role-Based Access Control (RBAC)

```
# Dictionary to store user permissions
permissions = {
    "customer": ["order_history", "menu_view"],
    "restaurant_owner": ["menu_edit", "order_history", "menu_view"],
    "admin": ["menu_edit", "order_history", "admin_dashboard"]
}

# Function to check access
def check_access(user_role, resource):
    if user_role in permissions:
        return resource in permissions[user_role]
    return False

# Test the function
print("Customer - Order History:",
      check_access("customer", "order_history"))

print("Customer - Admin Dashboard:",
```

```
    check_access("customer", "admin_dashboard"))


print("Restaurant Owner - Menu Edit:",
    check_access("restaurant_owner", "menu_edit"))


print("Restaurant Owner - Admin Dashboard:",
    check_access("restaurant_owner", "admin_dashboard"))


print("Admin - Admin Dashboard:",
    check_access("admin", "admin_dashboard"))
```

1. Output
2. Customer - Order History: True
3. Customer - Admin Dashboard: False
4. Restaurant Owner - Menu Edit: True
5. Restaurant Owner - Admin Dashboard: False
6. Admin - Admin Dashboard: True

3)Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Ans. Python Program: Two-Factor Authentication (2FA) – Paytm Example

This program simulates a two-factor authentication system like Paytm. First, the user enters a password. If the password is correct, a 6-digit OTP is generated. The user must enter the correct OTP to log in successfully.

Python Code

```
import random

# Set correct password
correct_password = "paytm123"

# Step 1: Enter password
password = input("Enter your password: ")

if password == correct_password:

    print("Password is correct!")

    # Step 2: Generate 6-digit OTP
    otp = random.randint(100000, 999999)

    print("Your OTP is:", otp)

    # Step 3: Enter OTP
    user_otp = int(input("Enter your OTP: "))
```

```
    # Step 4: Check OTP
    if user_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed")

else:
    print("Incorrect Password. Login Failed")
```

1. Sample Output
2. Enter your password: paytm123
3. Password is correct!
4. Your OTP is: 458921
5. Enter your OTP: 458921
6. Login Successful

Explanation

1. First, the user enters their password.
2. If the password is correct, the program generates a random 6-digit OTP.
3. The OTP is displayed in the console for testing.
4. The user enters the OTP.
5. If the OTP matches, the message Login Successful is displayed. Otherwise, login fails.

4)Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

Ans. JavaScript Program: Find Premium Users Active in the Last 7 Days

Code

```
const users = [
    { username: "Jeet", isPremium: true, lastLogin: "2024-06-10" },
    { username: "Rahul", isPremium: false, lastLogin: "2024-06-09" },
    { username: "Amit", isPremium: true, lastLogin: "2024-06-05" },
    { username: "Priya", isPremium: true, lastLogin: "2024-05-20" }
];


function getPremiumUsers(users) {
    return users.filter(user =>
        user.isPremium &&
        user.lastLogin >= "2024-06-03" &&
        user.lastLogin <= "2024-06-10"
    );
}


console.log(getPremiumUsers(users));
```

Output

```
[
```

```
  { username: "Jeet", isPremium: true, lastLogin: "2024-06-10" },
  { username: "Amit", isPremium: true, lastLogin: "2024-06-05" }
]
```

1. Explanation
2. First, we create a list of users.
3. We check whether the user has a premium account.
4. We check whether the user logged in between 3 June and 10 June 2024.
5. The filter() function selects only users who match both conditions.
6. Finally, it displays the selected users.

5)Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Ans. 5 Common Access Control Mistakes in Real-World Apps

No.	Access Control Mistake	Fix / Prevention Tip
1.	Weak passwords – Users use easy passwords.	Use strong passwords with letters, numbers, and symbols.
2.	Giving everyone admin access.	Give users only the permissions they need.
3.	Not checking user identity properly.	Always verify the user's login before allowing access.
4.	Allowing users to access other people's data.	Check user permissions before showing personal information.
5.	Not logging out inactive users.	Use automatic logout after a period of inactivity.